

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

1. IDENTIFICACIÓN DE LA GUÍA

Nombre de la guía:	Taller 1
Código de la guía (No.):	001
Taller(es) o Laboratorio(s) aplicable(s):	
Tiempo de trabajo práctico estimado:	10 horas
Asignatura(s) aplicable(s):	Programación Avanzada
Programa(s) Académico(s) / Facultad(es):	Ingeniería Electrónica / Ingeniería de Telecomunicaciones

COMPETENCIAS	CONTENIDO TEMÁTICO	INDICADOR DE LOGRO
C2: Capacidad para aplicar el diseño de ingeniería para generar soluciones que satisfagan necesidades específicas, considerando factores globales y económicos. C5: Capacidad para reconocer responsabilidades profesionales en situaciones de ingeniería y emitir juicios fundamentados al analizar e interpretar datos	Estructuras de control de flujo: Definición y uso de condicionales y bucles (ciclos). Estructuras de decisión: Implementación de condicionales simples y anidados para la validación de datos. Estructuras iterativas: Uso de ciclos para sumatorias, búsqueda de números primos y procesos iterativos de aproximación. Estructuras de datos: Creación, modificación y recorrido de listas. Uso de métodos básicos para agregar, eliminar, buscar y ordenar elementos.	Controla el flujo de ejecución de un programa empleando condicionales y ciclos para resolver problemas de ingeniería. Identifica los requerimientos necesarios para el desarrollo de un diseño o algoritmo en ingeniería. Utiliza listas y sus métodos básicos para almacenar, organizar y procesar conjuntos de datos en la solución de problemas de programación.

2. FUNDAMENTO TEÓRICO

El desarrollo de software y la automatización de procesos se han consolidado como competencias transversales e indispensables para los profesionales de Ingeniería Electrónica y de Ingeniería de Telecomunicaciones. En el entorno tecnológico actual, no basta con capturar datos; el ingeniero debe poseer la capacidad de manipular información y procesarla adecuadamente para la toma de decisiones informadas y el control de sistemas complejos. Para lograr esto, es imperativo dominar las estructuras de control de flujo, las cuales permiten que un algoritmo deje de ser una secuencia lineal de pasos y se convierta en una solución lógica capaz de reaccionar a diferentes escenarios. El uso de Python como lenguaje base responde a su versatilidad para la simulación de sistemas, el análisis de señales y la implementación de métodos numéricos.

En este taller, se busca que el estudiante utilice la lógica de programación para:

- Formular algoritmos que validen condiciones técnicas y geométricas específicas.
- Implementar procesos iterativos que permitan la resolución de problemas de programación científica, como la estimación de raíces cuadradas mediante métodos numéricos.
- Gestionar grandes volúmenes de datos de manera eficiente mediante el uso de bucles, preparando el terreno para el análisis de series de tiempo y señales de sensores que se abordará posteriormente en el curso.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

La integración de estas estructuras en entornos como Google Colab o Visual Studio Code proporciona al estudiante un ecosistema profesional para el diseño de soluciones que satisfagan necesidades de ingeniería bajo criterios de eficiencia y precisión.

3. OBJETIVO(S)

- Implementar estructuras condicionales para controlar el flujo de ejecución de un programa, permitiendo la toma de decisiones basada en la validación de datos y comparaciones lógicas.
- Desarrollar algoritmos iterativos mediante el uso de ciclos para la resolución de problemas de programación científica, tales como sumatorias complejas y procesos de aproximación numérica.
- Aplicar la lógica de programación en la simulación de escenarios reales.
- Traducir requerimientos técnicos en soluciones algorítmicas funcionales utilizando entornos de desarrollo profesional como Visual Studio Code o Google Colab.
- Utilizar listas para almacenar, organizar y procesar conjuntos de datos mediante estructuras iterativas y métodos propios de Python.

4. RECURSOS REQUERIDOS

Los elementos requeridos para realizar el taller son:

- Computador personal con internet.
- Acceso a software tipo Visual Studio Code, Google Colab, Kaggle Notebooks, entre otros.
- Material de estudio entregado en clase (Notebooks).

5. ASPECTOS DE SEGURIDAD

No aplica.

6. PROCEDIMIENTO O METODOLOGÍA PARA EL DESARROLLO

6.1. Resolver los siguientes ejercicios sobre la estructura condicional en Python:

- Escriba un programa que reciba un número y verifique si es divisible por 5 y 11, solo por 5, solo por 11 o por ninguno de los dos. Usar operadores lógicos y el módulo (%).
- Escriba un programa que lea un texto. Si su longitud es diferente de uno, muestre un mensaje de entrada inválida. En caso contrario, clasifique el carácter como vocal, consonante, número o carácter especial.
- Consultar para qué sirven los métodos que empiezan con "is", escribir "str.is" y observar la lista que se despliega. Escriba nuevamente el programa anterior utilizando isalpha() e isdigit() en las validaciones correspondientes. <https://python-reference.readthedocs.io/en/latest/docs/str/isalpha.html>
- Escriba un programa que reciba el número del mes (entre 1 y 12) y devuelva la cantidad de días que tiene el mes. Considere que es un año no bisiesto, y en caso de ingresar un número fuera del rango, el programa debe generar el mensaje "Mes inválido".
- Escriba un programa que reciba los ángulos de un triángulo y verifique si es válido o no (deben ser positivos y sumar 180 grados).
- Escriba un programa que reciba todos los lados de un triángulo y verifique si es válido o no (deben ser positivos y cumplir que la suma de cada par de lados sea mayor que el tercer lado).
- Diseñe un programa que permita calcular la factura de consumo de energía eléctrica. El consumo ingresado debe ser mayor o igual a cero. Si se ingresa un valor negativo, el programa debe informar

que el dato no es válido. Se recibe la cantidad de unidades kWh y la factura se calcula considerando las siguientes condiciones:

- Los primeros 50 kWh se facturan a 0.50/kWh.
- Para los siguientes 100 kWh, se facturan 0.75/kWh.
- Para los siguientes 100 kWh, se facturan 1.20/kWh.
- Por cada kWh adicional por encima de 250 kWh, se factura 1.50 por kWh.
- En toda factura se aplica un sobrecargo adicional del 20%.

Ejemplo: Para 80 unidades se calcula: Factura = $(50 \times 0.5 + 30 \times 0.75) \times 1.20$

6.2. Resolver los siguientes ejercicios sobre los ciclos en Python:

- Escriba un programa que reciba un número entero e imprima los cubos desde el 1 hasta el número ingresado.
- Escriba un programa que reciba tres números: “inicial”, “final” y “base”. La idea es sumar todos los números que están entre el número “inicial” y “final” que sean múltiplos de “base”, incluyendo los extremos del rango. Debe cumplirse $\text{inicial} \leq \text{final}$ y $\text{base} > 0$.
- Escriba un programa que reciba números hasta que se ingrese un número negativo. El programa debe devolver cual fue el número menor y el mayor que se ingresaron (sin incluir el número negativo que termina el ciclo). Si no se ingresa ningún número no negativo, el programa debe detectarlo y arrojar un mensaje de error.
- Escriba un programa que determine si un número ingresado es primo o no. Recuerde que un número primo es mayor que 1 y solo es divisible por 1 y por sí mismo.
AYUDA: Use un ciclo para buscar divisores y break cuando encuentre el primero. Los valores menores o iguales a 1 no son primos.
- Escriba un programa que reciba un número “inicial” y otro “final”. Almacenar los números que sean primos (reutilice la lógica desarrollada en el punto anterior).
- Escriba un programa que reciba un número entero mayor o igual que cero y calcule la factorial de dicho número, sin usar la función `math.factorial()` o similares. Recuerde que $0! = 1$.
- Escriba un algoritmo que estime la raíz cuadrada de cualquier número real positivo S por medio del siguiente proceso iterativo:

$$x_{k+1} = \frac{1}{2} \left(x_k + \frac{S}{x_k} \right)$$

El programa debe imprimir cuántas iteraciones necesitó para converger.

AYUDA:

- Debe definir un valor de partida para el algoritmo (es el primer valor de x_k y debe ser mayor que 0).
- Calcule sucesivamente el siguiente valor de la aproximación x_{k+1} .
- El programa debe terminar cuando $|x_{k+1} - x_k| \leq 10^{-3}$ (Disminuirlo si requiere mayor precisión).

6.3. Ciclos y listas

- Escriba un programa que genere 20 números enteros aleatorios entre 0 y 100 y los almacene uno por uno en una lista.
- Recorra la lista generada en el ejercicio anterior y construya dos listas nuevas, una con los números pares y otra con los impares. Conserve el orden y los valores repetidos.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

- c) Escriba un programa que recorra una lista numérica y construya otra lista que guarde la suma acumulada después de cada elemento. No use librerías externas.
- d) Escribir un programa que genere un vector aleatorio (ver ayuda) y un número entre 0 y 1 (ver ejemplos de clase) como umbral por parte del usuario. Devolver dos listas, una con los números mayores al umbral y la otra lista con los números menores o iguales al umbral.

AYUDA: Usar los siguientes comandos para generar el vector aleatorio:

```
import numpy as np
rng = np.random.default_rng(10)
vector = rng.random(20)
```

6.4. Ejercicios integradores

- a) En una estación de monitoreo de Parque i, se recibe una lista de números flotantes que representan mediciones de voltaje tomadas de un sensor de alta precisión. Los valores de voltajes medidos fueron 4.2, 5.1, 0.0, 3.8, -0.5, 4.9, 5.0 y 1.2 V. Se requiere diseñar un programa que realice las siguientes tareas:
- Contar cuántas veces el voltaje es mayor o igual a 5.0 V (límite superior de seguridad) o menor o igual a 0.0 V (posible falla de conexión).
 - Crear una nueva lista que contenga únicamente los valores de voltaje que están dentro del rango de operación normal (valores mayores a 0.0 V y menores a 5.0 V).
 - El programa debe imprimir:
 - La cantidad total de alertas detectadas.
 - La lista con los voltajes en el rango normal.
 - El promedio de los voltajes que se encuentran en el rango normal (si la lista está vacía, debe indicarlo).
- b) En un centro de datos de la Facultad de Ingeniería del ITM, se requiere un programa para registrar manualmente las temperaturas de los racks de servidores. El programa debe permitir al usuario ingresar las mediciones una a una bajo las siguientes condiciones:
- Utilizar un ciclo while para solicitar al usuario que ingrese valores de temperatura (enteros) mediante la función input(). El proceso de entrada debe finalizar cuando el usuario ingrese el valor -99 (este valor es una señal de parada y no debe incluirse en la lista de datos ni en los cálculos).
 - Cada temperatura válida debe ser agregada a una lista llamada registro_temp.
 - Una vez finalizada la captura, el programa debe:
 - Imprimir la lista completa de temperaturas capturadas.
 - Calcular y mostrar la temperatura máxima registrada (sin utilizar la función max() de Python).
 - Contar cuántas veces se superó el umbral crítico de 35°C.

NOTA: Si el primer valor ingresado es -99, el programa debe mostrar un mensaje indicando que no se registraron temperaturas y, por tanto, no es posible calcular la temperatura máxima.

- c) En un proyecto de telemetría del ITM, un ingeniero electrónico gestiona una lista de identificadores (IDs) de sensores que están transmitiendo datos en una frecuencia específica. La lista inicial de sensores

 Institución Universitaria Reacreditada en Alta Calidad	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

activos es: [105, 202, 105, 305, 402, 501]. Diseñe un programa que realice las siguientes operaciones de forma secuencial, utilizando los métodos de listas vistos en clase:

- i. Un nuevo sensor con ID 608 se ha conectado a la red. Agréguelo al final de la lista.
- ii. El sensor con ID 999 es de alta prioridad. Insértelo en la segunda posición (índice 1) de la lista.
- iii. El sensor con ID 305 ha fallado. Elimínelo de la lista usando su valor.
- iv. El último sensor de la lista se ha desconectado. Elimínelo usando el método adecuado para extraer el último elemento.
- v. Determine y muestre cuántas veces aparece el ID 105 en la lista.
- vi. Encuentre y muestre la posición (índice) en la que se encuentra el sensor 402.
- vii. Ordene la lista de sensores de menor a mayor para facilitar el escaneo de frecuencias y muestre el resultado final.

7. PARÁMETROS PARA ELABORACIÓN DEL INFORME

No aplica

8. DISPOSICIÓN DE RESIDUOS

No aplica

9. BIBLIOGRAFÍA

- [1] <https://github.com/estebangonzalezITM/ProgramacionAvanzada>
[2] <https://python-reference.readthedocs.io/en/latest/docs/str/isalpha.html>

Elaborado por:	Esteban Gonzalez Valencia
Revisado por:	
Versión:	1
Fecha:	Agosto de 2026